

ios Deep Dive And Build Manual

- [iOS Deep-Dive & Build Manual](#)
 - [1. Architecture Overview](#)
 - [1.1 High-Level Structure](#)
 - [1.2 NetworkPeerCore \(SPM Package\)](#)
 - [1.3 UI Layer \(Xcode Project\)](#)
 - [2. Cognito Custom Auth Integration](#)
 - [2.1 Wire Format \(Backend Contract\)](#)
 - [2.2 NetworkPeerAPI.swift — Key Methods](#)
 - [2.3 AuthSession.swift — Token Management](#)
 - [3. Local Development & Physical Device Testing](#)
 - [3.1 Prerequisites](#)
 - [3.2 Xcode Project Setup](#)
 - [3.3 Network Configuration for Local Backend](#)
 - [3.4 Build & Run on Physical Device](#)
 - [3.5 Free vs Paid Developer Account](#)
 - [4. Evidence Upload Pipeline \(S3 Presigned\)](#)
 - [4.1 Flow](#)
 - [4.2 EvidenceUploader.swift Implementation](#)
 - [5. Push Notifications \(APNs\)](#)
 - [5.1 Capabilities Setup](#)
 - [5.2 Device Token Registration](#)
 - [5.3 APNs Auth \(Backend\)](#)
 - [6. CI/CD & App Store Deployment](#)
 - [6.1 Fastlane Setup \(Optional but Recommended\)](#)
 - [6.2 xcodebuild Commands \(No Fastlane\)](#)
 - [6.3 ExportOptions Plists](#)
 - [6.4 Signing Certificates](#)
 - [7. Testing](#)
 - [7.1 Unit & Contract Tests](#)
 - [7.2 Test Coverage](#)

- [7.3 UI Tests](#)
- [8. Troubleshooting](#)
- [9. Quick Reference Commands](#)

iOS Deep-Dive & Build Manual

NetworkPeer iOS Application — Native Swift 6 / Swift Package Manager

1. Architecture Overview

1.1 High-Level Structure

```
apps/ios/
├── NetworkPeer.xcodeproj/      # Xcode project for UI layer (SwiftUI)
├── Package.swift              # Swift Package Manager manifest
├── Sources/
│   ├── NetworkPeerCore/      # SPM package: business logic, networking, models
│   │   ├── NetworkPeerAPI.swift # API client (Cognito auth, jobs, evidence, sockets)
│   │   ├── NetworkPeerModels.swift # Codable models (snake_case JSON)
│   │   ├── AuthSession.swift   # Token storage, refresh, Keychain integration
│   │   ├── EvidenceUploader.swift # Presigned S3 upload pipeline
│   │   └── DataOnlyPushPayload.swift
│   ├── Tests/
│   │   └── NetworkPeerCoreTests/ # Contract & unit tests (swift test)
│   └── NetworkPeer/          # SwiftUI App + Views
│       ├── NetworkPeerApp.swift # App entry point
│       └── Features/          # Feature modules (Jobs, Auth, Evidence, etc.)
└── Config.*.xcconfig         # Build configurations (Debug/Release)
```

1.2 NetworkPeerCore (SPM Package)

Target: `NetworkPeerCore` — pure Swift, no UIKit/SwiftUI dependencies.

Module	Responsibility
<code>NetworkPeerAPI.swift</code>	All HTTP + Socket.IO calls; Cognito Custom Auth flows; request/response mapping
<code>NetworkPeerModels.swift</code>	<code>Codable</code> DTOs matching backend OpenAPI spec; <code>snake_case</code> → <code>camelCase</code> via <code>CodingKeys</code>
<code>AuthSession.swift</code>	In-memory access token + Keychain-stored refresh token; automatic refresh on 401
<code>EvidenceUploader.swift</code>	3-step S3 presigned upload: reserve → PUT → complete
<code>DataOnlyPushPayload.swift</code>	Silent push deserialization for background sync

Dependencies: Zero external deps (Foundation only). Uses `URLSession`, `Combine` (async/await), `CryptoKit`.

1.3 UI Layer (Xcode Project)

- **Framework:** SwiftUI + `@tanstack/react-start` parity patterns
- **State:** `ObservableObject` view models + `@Published` properties
- **Navigation:** `NavigationStack` with path-based deep linking
- **Authentication:** `AuthViewModel` wraps `AuthSession`; publishes `isAuthenticated`, `userRole`

2. Cognito Custom Auth Integration

2.1 Wire Format (Backend Contract)

```
// Request OTP
struct OTPRequest: Encodable {
    let phone_number: String
    let role: UserRole        // "CLIENT" | "WORKER"
}

// Response (snake_case from backend)
struct OTPResponse: Decodable {
    let challenge_id: String
    let expires_in_seconds: Int
    let otp_length: Int
    let delivery: DeliveryInfo
}

struct DeliveryInfo: Decodable {
    let transport: String    // "sms"
    let to: String
}

// Verify OTP
struct OTPVerifyRequest: Encodable {
    let phone_number: String
    let challenge_id: String
    let otp: String
    let transport: String    // "sms" | "browser" (mobile uses "sms")
}

// Response
struct TokenResponse: Decodable {
    let access_token: String
    let refresh_token: String
    let id_token: String
    let expires_in: Int
    let user: UserProfile
}
```

2.2 NetworkPeerAPI.swift — Key Methods

```
// MARK: - Authentication
func requestOTP(phone: String, role: UserRole) async throws -> OTPResponse
func verifyOTP(challengeId: String, otp: String, transport: String) async
    throws -> TokenResponse
func refreshAccessToken() async throws -> TokenResponse
func logout(refreshToken: String) async throws

// MARK: - Jobs
func listJobs(cursor: String?, filters: JobFilters) async throws ->
    PaginatedResponse<Job>
func createJob(_ request: CreateJobRequest) async throws -> Job
func acceptJob(id: String) async throws -> Job
func completeJob(id: String) async throws -> Job

// MARK: - Evidence (3-step S3 upload)
func reserveEvidenceUpload(_ request: EvidenceReserveRequest) async throws ->
    EvidenceReservation
func uploadEvidenceData(_ data: Data, to reservation: EvidenceReservation)
    async throws
func completeEvidenceUpload(_ request: EvidenceCompleteRequest) async throws
    -> Evidence

// MARK: - Real-time
func connectSocket(token: String) -> SocketIOClient
func subscribeToJob(_ jobId: String, handler: @escaping (JobEvent) -> Void)
```

2.3 AuthSession.swift — Token Management

```
final class AuthSession: ObservableObject {
    @Published var accessToken: String?
    @Published var refreshToken: String? // Persisted to Keychain
    @Published var user: UserProfile?

    // Keychain keys
    private let refreshTokenKey = "networkpeer.refresh_token"
    private let userProfileKey = "networkpeer.user_profile"

    // Auto-refresh on 401
    func authenticatedRequest<T>(<_ request: () async throws -> T) async throws
        -> T

    // Keychain operations (kSecAttrAccessibleWhenUnlockedThisDeviceOnly)
    func saveRefreshToken(_ token: String) throws
    func loadRefreshToken() -> String?
    func clearTokens()
}
```

3. Local Development & Physical Device Testing

3.1 Prerequisites

Tool	Version	Install
Xcode	15.4+	Mac App Store
iOS Device	17.0+	Physical iPhone
Apple ID	Free tier OK	developer.apple.com

3.2 Xcode Project Setup

```
cd /Users/adityasharma/Desktop/NETWORKPEER/apps/ios
open NetworkPeer.xcodeproj
```

Signing & Capabilities Configuration: 1. Select **NetworkPeer** target → **Signing & Capabilities** 2. ☒ **Automatically manage signing** 3. **Team:** Your Apple ID (Personal Team) 4. **Bundle Identifier:** **Must be unique** — change from `com.networkpeer.app` to `com.<yourname>.networkpeer` 5. **Capabilities:** Add **Push Notifications** + **Background Modes** → **Remote notifications**

3.3 Network Configuration for Local Backend

Info.plist (or `NetworkPeer/Info.plist`):

```
<key>NSAppTransportSecurity</key>
<dict>
  <key>NSAllowsArbitraryLoads</key>
  <true/>
  <key>NSExceptionDomains</key>
  <dict>
    <key>localhost</key>
    <dict>
      <key>NSExceptionAllowsInsecureHTTPLoads</key>
      <true/>
    </dict>
  </dict>
</dict>
```

NetworkPeerAPI.swift — Toggle base URL:

```
// Production
private let baseURL = URL(string: "https://api.networkpeer.com")!

// Local development (your Mac's LAN IP)
private let baseURL = URL(string: "http://192.168.1.XXX:3000")!
// Find IP: ifconfig | grep "inet " | grep -v 127.0.0.1
```


3.4 Build & Run on Physical Device

```
# 1. Connect iPhone via USB → Unlock → Trust computer
# 2. In Xcode: Select your iPhone from device dropdown (top-left)
# 3. Press ⌘R (Run) or click ▶
# 4. First build: 2-3 minutes (SPM resolution)
# 5. App installs on device

# 6. On iPhone: Settings → General → VPN & Device Management
#    → Tap your Apple ID → Trust → Confirm
```

3.5 Free vs Paid Developer Account

Feature	Free (Personal Team)	Paid (\$99/yr)
Device testing	✅ 7-day expiry	✅ 1 year
TestFlight	❌	✅
App Store	❌	✅
Push Notifications	✅ (limited)	✅ Full
Multiple devices	3 max	Unlimited

For demo: Free account works. Rebuild every 7 days.

4. Evidence Upload Pipeline (S3 Presigned)

4.1 Flow

```
1. Client → POST /evidence/reserve { job_id, media_type, mime_type, file_size }
2. Backend → Returns { reservation_id, upload: { url, fields } }
3. Client → PUT direct to S3 (multipart/form-data with fields + file)
4. Client → POST /evidence/complete { reservation_id, s3_key }
5. Backend → Verifies S3 object → Creates evidence record
```

4.2 EvidenceUploader.swift Implementation

```

func uploadEvidence(
    jobId: String,
    subtaskId: String?,
    mediaType: MediaType,
    mimeType: String,
    fileData: Data,
    capturedAt: Date
) async throws -> Evidence {
    // Step 1: Reserve
    let reservation = try await api.reserveEvidenceUpload(
        EvidenceReserveRequest(
            jobId: jobId,
            subtaskId: subtaskId,
            mediaType: mediaType,
            mimeType: mimeType,
            fileSizeBytes: fileData.count
        )
    )

    // Step 2: Direct S3 upload
    var request = URLRequest(url: reservation.uploadURL)
    request.httpMethod = "POST"
    let boundary = "Boundary-\(UUID().uuidString)"
    request.setValue("multipart/form-data; boundary=\(boundary)",
        forHTTPHeaderField: "Content-Type")

    var body = Data()
    // Add policy fields
    for (key, value) in reservation.fields {
        body.append("--\((boundary)\r\n")
        body.append("Content-Disposition: form-data; name=\"\((key)\""\r\n\r\n")
        body.append("\((value)\r\n")
    }
    // Add file
    body.append("--\((boundary)\r\n")
    body.append("Content-Disposition: form-data; name=\"file\";
        filename=\"evidence\""\r\n")
    body.append("Content-Type: \((mimeType)\r\n\r\n")
    body.append(fileData)
    body.append("\r\n--\((boundary)--\r\n")

```

```

request.httpBody = body

let (_, response) = try await URLSession.shared.data(for: request)
guard (response as? HTTPURLResponse)?.statusCode == 204 else {
    throw UploadError.s3UploadFailed
}

// Step 3: Complete
return try await api.completeEvidenceUpload(
    EvidenceCompleteRequest(
        reservationId: reservation.id,
        s3Key: reservation.s3Key,
        capturedAt: capturedAt
    )
)
}

```

5. Push Notifications (APNs)

5.1 Capabilities Setup

1. Xcode → Target → **Signing & Capabilities** → + **Capability**
2. Add **Push Notifications**
3. Add **Background Modes** →  **Remote notifications**

5.2 Device Token Registration

```

// In NetworkPeerApp.swift or AppDelegate
func application(_ application: UIApplication,
    didRegisterForRemoteNotificationsWithDeviceToken deviceToken: Data) {
    let token = deviceToken.map { String(format: "%02.2hhx", $0) }.joined()
    Task {
        try await api.registerDeviceToken(token, platform: "APNS")
    }
}

```

5.3 APNs Auth (Backend)

- Backend uses **token-based auth** (`.p8` key + Key ID + Team ID)
 - Stored in AWS Secrets Manager → Injected into ECS task env vars
-

6. CI/CD & App Store Deployment

6.1 Fastlane Setup (Optional but Recommended)

```
cd /Users/adityasharma/Desktop/NETWORKPEER/apps/ios
gem install fastlane
fastlane init
```

Fastfile (`fastlane/Fastfile`):

```
default_platform(:ios)

platform :ios do
  lane :build_dev do
    build_app(
      scheme: "NetworkPeer",
      configuration: "Debug",
      export_method: "development",
      output_directory: "./build",
      output_name: "NetworkPeer-Dev.ipa"
    )
  end

  lane :build_release do
    build_app(
      scheme: "NetworkPeer",
      configuration: "Release",
      export_method: "app-store",
      output_directory: "./build",
      output_name: "NetworkPeer-Release.ipa"
    )
  end

  lane :testflight do
    build_release
    upload_to_testflight(
      api_key_path: "./fastlane/app_store_connect_api_key.json",
      skip_waiting_for_build_processing: true
    )
  end
end
```

6.2 xcodebuild Commands (No Fastlane)

```
# Development IPA (for device testing)
xcodebuild -project NetworkPeer.xcodeproj \
  -scheme NetworkPeer \
  -configuration Debug \
  -destination generic/platform=iOS \
  -archivePath ./build/NetworkPeer-Dev.xcarchive \
  archive

xcodebuild -exportArchive \
  -archivePath ./build/NetworkPeer-Dev.xcarchive \
  -exportOptionsPlist ExportOptions-Development.plist \
  -exportPath ./build

# Release Archive (for TestFlight/App Store)
xcodebuild -project NetworkPeer.xcodeproj \
  -scheme NetworkPeer \
  -configuration Release \
  -destination generic/platform=iOS \
  -archivePath ./build/NetworkPeer-Release.xcarchive \
  archive

xcodebuild -exportArchive \
  -archivePath ./build/NetworkPeer-Release.xcarchive \
  -exportOptionsPlist ExportOptions-AppStore.plist \
  -exportPath ./build
```

6.3 ExportOptions Plists

ExportOptions-Development.plist:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/
  DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>method</key><string>development</string>
  <key>teamID</key><string>YOUR_TEAM_ID</string>
  <key>signingStyle</key><string>automatic</string>
  <key>stripSwiftSymbols</key><true/>
  <key>compileBitcode</key><false/>
</dict>
</plist>
```

ExportOptions-AppStore.plist: Change `method` to `app-store` .

6.4 Signing Certificates

Environment	Certificate	Profile
Development	iOS Development	Automatic (Xcode)
Ad Hoc	iOS Distribution	Manual (for TestFlight internal)
App Store	iOS Distribution	Automatic (App Store Connect)

Generate via Xcode: Preferences → Accounts → Manage Certificates → + → iOS Development / Distribution

7. Testing

7.1 Unit & Contract Tests

```
cd /Users/adityasharma/Desktop/NETWORKPEER/apps/ios
swift test
# Output: 18 tests across 3 suites (NetworkPeer mobile contract, Device
  registration, Authentication refresh)
```


7.2 Test Coverage

```
swift test --enable-code-coverage
xcrun llvm-cov export -format="lcov" .build/debug/
  NetworkPeerCorePackageTests.xctest/Contents/MacOS/
  NetworkPeerCorePackageTests -instr-profile .build/debug/codecov/
  default.profdata > coverage.lcov
```

7.3 UI Tests

```
xcodebuild test \
  -project NetworkPeer.xcodeproj \
  -scheme NetworkPeer \
  -destination 'platform=iOS Simulator,name=iPhone 15 Pro,OS=latest' \
  -derivedDataPath ./DerivedData
```

8. Troubleshooting

Issue	Solution
“No signing certificate”	Xcode → Settings → Accounts → + → Apple ID → Manage Certificates → + → iOS Development
“Could not launch”	Delete app from iPhone → Clean Build Folder (⌘ ↑ K) → Rebuild
SPM resolution fails	File → Packages → Reset Package Caches → Resolve
Push not working	Verify APNs key in AWS Secrets Manager; check Background Modes capability
Network error on local backend	Verify <code>NSAppTransportSecurity</code> in Info.plist; check Mac firewall (port 3000)
“Bundle identifier not available”	Change to unique ID: <code>com.yourname.networkpeer</code>

9. Quick Reference Commands

```
# Open project
open /Users/adityasharma/Desktop/NETWORKPEER/apps/ios/NetworkPeer.xcodeproj

# Run tests
cd /Users/adityasharma/Desktop/NETWORKPEER/apps/ios && swift test

# Clean build
cd /Users/adityasharma/Desktop/NETWORKPEER/apps/ios && xcodebuild clean
    -project NetworkPeer.xcodeproj -scheme NetworkPeer

# Build development IPA
cd /Users/adityasharma/Desktop/NETWORKPEER/apps/ios && fastlane build_dev

# Build release + TestFlight
cd /Users/adityasharma/Desktop/NETWORKPEER/apps/ios && fastlane testflight

# Find Mac LAN IP for local backend
ifconfig | grep "inet " | grep -v 127.0.0.1
```

Last Updated: 2026-09-03 | NetworkPeer v1.3 | Swift 6 / Xcode 15.4+